

Serverless Architecture for Flash Sale – Handling Concurrency in

*A hands-on session by, **Harrish Ragavendar***

AWS Community Builder - Serverless

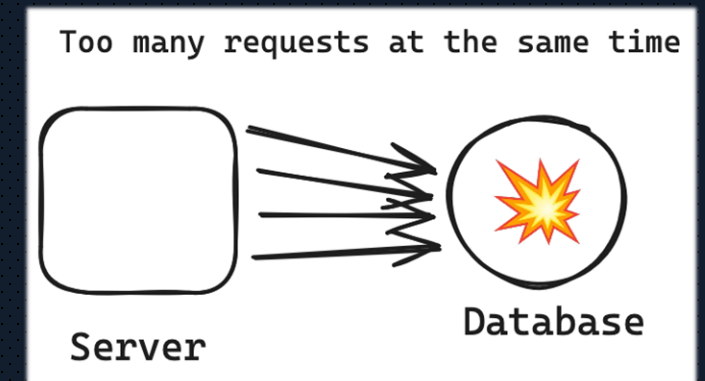
Agenda

- Design a high-concurrency flash sale system and understand where race conditions occur.
- Configure Amazon DynamoDB to manage inventory using atomic conditional writes.
- Build HTTP APIs using Amazon API Gateway integrated with AWS Lambda.
- Implement race-condition-safe inventory deduction without using distributed locks.
- Simulate high-concurrency purchase traffic and observe DynamoDB's consistency guarantees.
- Analyze how DynamoDB scales to handle thousands of concurrent requests safely.

GitHub URL: <https://github.com/harrishragavendar/AWS-FlashSale-Concurrency-Demo>

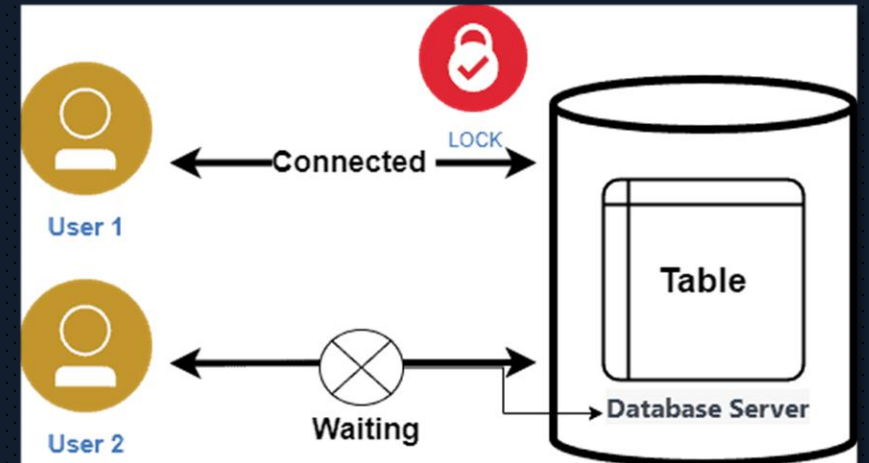
The Problem - Why Flash Sales Hit Hard

- Thousands of users click “Buy” at the same time
- Inventory is limited (e.g. 100 items)
- Traditional systems oversell under load
- **Question** - How do we guarantee correctness when requests arrive concurrently?



Traditional Solution

- Use database transactions with row-level locks to control concurrent updates
- Read inventory, validate availability, then update within a transaction
- Requires careful tuning of isolation levels to avoid inconsistencies
- Suffers from lock contention, deadlocks, and scalability limits under high load



Serverless Solution Using DynamoDB

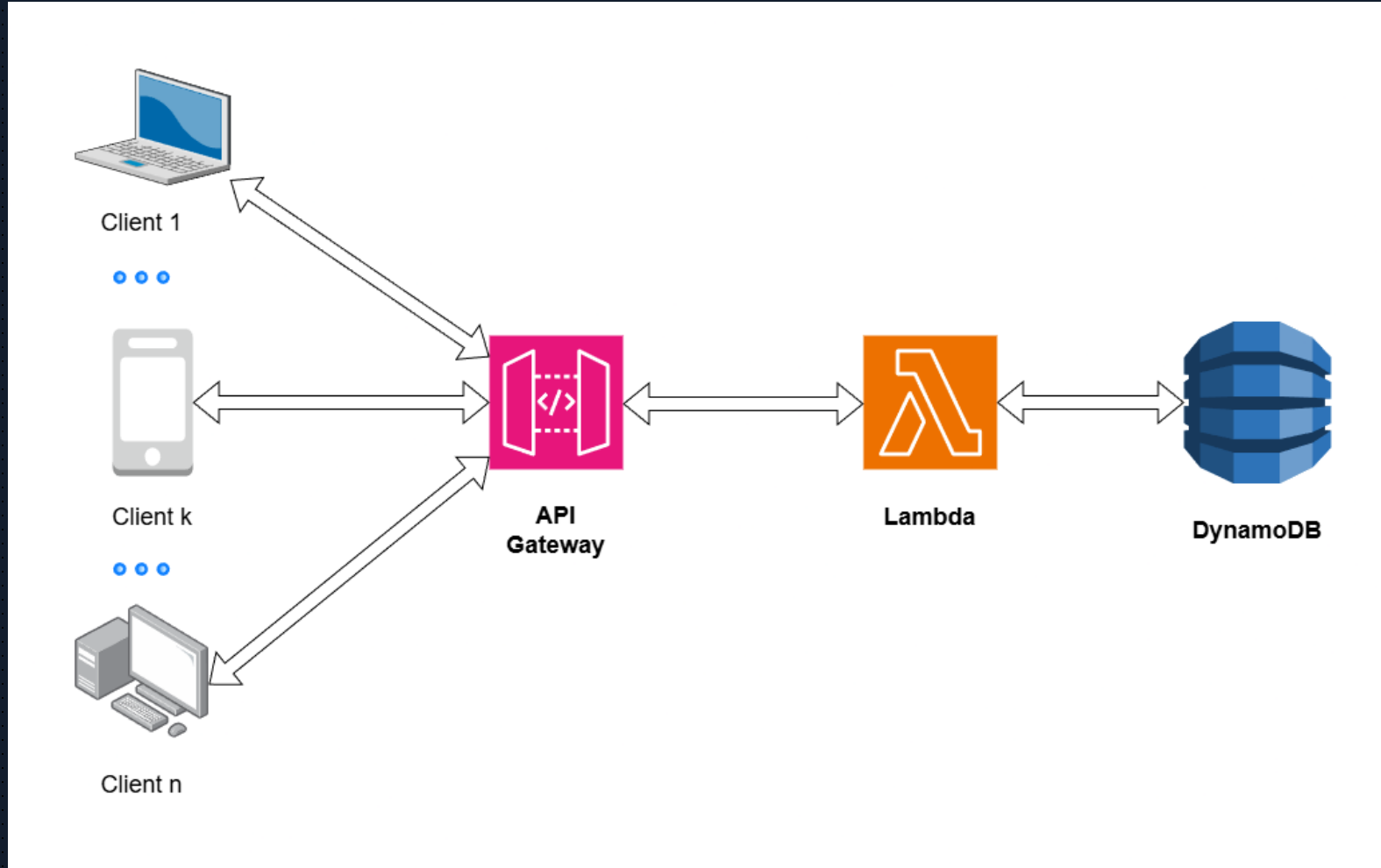
- Use DynamoDB conditional writes to update inventory atomically
- Combine condition check and update in a single `UpdateItem` operation
- Avoid application-level locks and explicit transaction management
- Execute safely at scale using AWS Lambda and API Gateway



Advantages of DynamoDB Over Traditional Solution

Aspect	Traditional Approach	Serverless Approach
Concurrency Model	Pessimistic locking or transactions	Optimistic concurrency with conditional writes
Request Behaviour	Requests wait for locks to be released	Requests are evaluated immediately
Conflict Handling	Blocking, retries, or deadlocks	Fail-fast conditional check failures
Scalability Under Load	Limited by locks and connection pools	Automatically scales for burst traffic

Architecture Diagram



Connect with me!



Harrish Ragavendar Srinivasan
4 X AWS Certified | Member Technical
@ Zentropy | Backend Developer

